Het Patel IIT2024229

Assignment III

Pre-requisites

Since Assignment 3 requires columns that were not present in the Lab 2 schema (specifically `grade` in the `Enrolment` table and `total_credits` in the `Student` table), I am adding these attributes into my tables first to align it with the new requirements.

```
ALTER TABLE Enrolment ADD grade CHAR(1);
ALTER TABLE Student ADD total_credits INT DEFAULT 0;
```

1. Calculate and Update Grades

```
DELIMITER //

CREATE PROCEDURE UpdateGrades()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_sid INT;
    DECLARE v_cid INT;
    DECLARE v_marks INT;
    DECLARE v_grade CHAR(1);
    DECLARE rows_updated INT DEFAULT 0;

    DECLARE cur_enroll CURSOR FOR
    SELECT student_id, course_id, marks FROM Enrolment;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_enroll;

    read_loop: LOOP
    FETCH cur_enroll INTO v_sid, v_cid, v_marks;
    IF done THEN
    LEAVE read_loop;
    END IF;
```

```
        IF v_marks >= 90 THEN SET v_grade = 'A';
        ELSEIF v_marks >= 75 THEN SET v_grade = 'B';
        ELSEIF v_marks >= 60 THEN SET v_grade = 'C';
        ELSEIF v_marks >= 40 THEN SET v_grade = 'D';
        ELSE SET v_grade = 'F';
        END IF;

        UPDATE Enrolment
        SET grade = v_grade
        WHERE student_id = v_sid AND course_id = v_cid;

        SET rows_updated = rows_updated + 1;
        END LOOP;

        CLOSE cur_enroll;

        SELECT CONCAT('Total rows updated: ', rows_updated) AS Result;
END //

DELIMITER ;
```

#	student_ic	course_ic	semester	marks	grade
1	1	10	1	92	A
2	1	11	1	88	B
3	1	18	1	85	B
4	1	19	1	90	A
5	2	10	1	95	A
6	2	11	1	91	A
7	3	12	1	75	B
8	3	13	1	80	B
9	4	14	1	60	C
10	6	12	1	85	B
11	8	17	1	98	A
12	9	10	1	70	C
13	10	12	1	78	B
14	12	10	1	82	B
*	NULL	NULL	NULL	NULL	NULL

2. Find Department Topper

```
DELIMITER //

CREATE PROCEDURE FindDeptToppers()
BEGIN
	DECLARE done INT DEFAULT FALSE;
	DECLARE v_dept_id INT;
	DECLARE v_dept_name VARCHAR(50);
	DECLARE v_student_name VARCHAR(100);
	DECLARE v_cgpa DECIMAL(4,2);

	DECLARE cur_dept CURSOR FOR SELECT dept_id, dept_name FROM
Department;
	DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

	CREATE TEMPORARY TABLE IF NOT EXISTS DeptToppers (
	Department_ID INT,
	Student_Name VARCHAR(100),
	CGPA VARCHAR(50)
	);

	OPEN cur_dept;

	read_loop: LOOP
	FETCH cur_dept INTO v_dept_id, v_dept_name;
	IF done THEN
	LEAVE read_loop;
	END IF;

	SET v_student_name = NULL;
	SET v_cgpa = NULL;

	SELECT CONCAT(first_name, ' ', last_name), cgpa
	INTO v_student_name, v_cgpa
	FROM Student
	WHERE dept_id = v_dept_id
	ORDER BY cgpa DESC
	LIMIT 1;

	IF v_student_name IS NOT NULL THEN
	INSERT INTO DeptToppers VALUES (v_dept_id, v_student_name, v_cgpa);
	ELSE
```

```
        INSERT INTO DeptToppers VALUES (v_dept_id, 'No students in department',
'N/A');
        END IF;

        END LOOP;

        CLOSE cur_dept;

        SELECT * FROM DeptToppers;
        DROP TEMPORARY TABLE DeptToppers;
END //

DELIMITER ;
```

#	Department_ID	Student_Name	CGPA
1	101	Amit Sharma	9.20
2	102	Eva Green	8.00
3	103	Charlie Chaplin	6.80
4	104	Frank Castle	5.50
5	105	Grace Hopper	9.50

3. Trigger for Credit Limit

```
DELIMITER //

CREATE TRIGGER check_credit_limit
BEFORE INSERT ON Enrolment
FOR EACH ROW
BEGIN
        DECLARE v_current_credits INT DEFAULT 0;
        DECLARE v_new_course_credits INT DEFAULT 0;

        SELECT credits INTO v_new_course_credits
        FROM Course WHERE course_id = NEW.course_id;

        SELECT IFNULL(SUM(c.credits), 0) INTO v_current_credits
        FROM Enrolment e
        JOIN Course c ON e.course_id = c.course_id
        WHERE e.student_id = NEW.student_id;

        IF (v_current_credits + v_new_course_credits) > 30 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Total credits cannot exceed 30.';
        END IF;
END //

DELIMITER ;
```

4. Scholarship Eligibility Function

```
DELIMITER //

CREATE FUNCTION is_scholarship_eligible(p_student_id INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
        DECLARE v_cgpa DECIMAL(4,2);
        DECLARE v_dept_id INT;
        DECLARE v_avg_cgpa DECIMAL(4,2);
        DECLARE v_fail_count INT;

        SELECT cgpa, dept_id INTO v_cgpa, v_dept_id
        FROM Student WHERE student_id = p_student_id;

        SELECT AVG(cgpa) INTO v_avg_cgpa
        FROM Student WHERE dept_id = v_dept_id;

        SELECT COUNT(*) INTO v_fail_count
        FROM Enrolment WHERE student_id = p_student_id AND grade = 'F';

        IF v_cgpa > v_avg_cgpa AND v_fail_count = 0 THEN
        RETURN 1;
        ELSE
        RETURN 0;
        END IF;
END //

DELIMITER ;
```

#	student_ic	first_name	last_name	Eligible
1	1	Alice	Brown	1
2	2	Amit	Sharma	1
3	3	Bob	Marley	0
4	4	Charlie	Chaplin	0
5	6	Eva	Green	1
6	7	Frank	Castle	0
7	8	Grace	Hopper	0
8	9	Harry	Potter	0
9	10	Irene	Adler	0
10	12	Arun	Kumar	0

5. Optimize Salary Procedure

```
DELIMITER //

CREATE PROCEDURE optimize_salary()
BEGIN
        UPDATE Faculty f
        JOIN (
        SELECT dept_id, AVG(salary) as avg_sal
        FROM Faculty
        GROUP BY dept_id
        ) avg_table ON f.dept_id = avg_table.dept_id
        SET f.salary = f.salary * 1.15
        WHERE f.salary < avg_table.avg_sal;

        SELECT ROW_COUNT() AS 'Updated_Rows';
END //

DELIMITER ;
```

#	Updated_Rows
1	4

6. Department Topper Function

```
DELIMITER //
```

```
CREATE FUNCTION get_department_topper(p_dept_id INT)
RETURNS VARCHAR(100)
DETERMINISTIC
READS SQL DATA
BEGIN
        DECLARE v_name VARCHAR(100);

        SELECT CONCAT(first_name, ' ', last_name) INTO v_name
        FROM Student
        WHERE dept_id = p_dept_id
        ORDER BY cgpa DESC
        LIMIT 1;

        RETURN IFNULL(v_name, 'No Student Found');
END //

DELIMITER ;
```

#	dept_id	dept_name	Topper
1	101	CSE	Amit Sharma
2	102	ECE	Eva Green
3	103	MECH	Charlie Chaplin
4	104	CIVIL	Frank Castle
5	105	EE	Grace Hopper

7. Nested Loops Display

```
DELIMITER //

CREATE PROCEDURE DeptCourseStats()
BEGIN
        DECLARE done_dept INT DEFAULT FALSE;
        DECLARE v_dept_id INT;

        DECLARE cur_dept CURSOR FOR SELECT dept_id FROM Department;
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET done_dept = TRUE;

        CREATE TEMPORARY TABLE IF NOT EXISTS Report (
        Details VARCHAR(255)
        );
```

```
        OPEN cur_dept;
        dept_loop: LOOP
        FETCH cur_dept INTO v_dept_id;
        IF done_dept THEN LEAVE dept_loop; END IF;

        INSERT INTO Report VALUES (CONCAT('Department: ', v_dept_id));

        BEGIN
        DECLARE done_course INT DEFAULT FALSE;
        DECLARE v_course_name VARCHAR(100);
        DECLARE v_course_id INT;
        DECLARE v_count INT;

        DECLARE cur_course CURSOR FOR
                SELECT course_id, course_name FROM Course WHERE dept_id =
v_dept_id;
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET done_course = TRUE;

        OPEN cur_course;
        course_loop: LOOP
                FETCH cur_course INTO v_course_id, v_course_name;
                IF done_course THEN LEAVE course_loop; END IF;

                SELECT COUNT(*) INTO v_count FROM Enrolment WHERE course_id =
v_course_id;

                INSERT INTO Report VALUES (
                CONCAT('   Course: ', v_course_name, ' -> Total Students: ', v_count)
                );
        END LOOP;
        CLOSE cur_course;
        END;
        END LOOP;
        CLOSE cur_dept;

        SELECT * FROM Report;
        DROP TEMPORARY TABLE Report;
END //

DELIMITER ;
```

#	Details
1	Department: 105
2	Department: 104
3	Department: 103
4	Department: 102
5	Department: 101
6	Course: Thermodynamics -> Total Students: 1
7	Course: Structures -> Total Students: 0
8	Course: Signals -> Total Students: 1
9	Course: OS -> Total Students: 1
10	Course: Fluid Mech -> Total Students: 0
11	Course: Digital Logic -> Total Students: 3
12	Course: DBMS -> Total Students: 1
13	Course: Data Structures -> Total Students: 4
14	Course: Circuits -> Total Students: 1
15	Course: Algorithms -> Total Students: 2